

SMART HOSPITAL IoT-BASED MONITORING SYSTEM

Internship Project Report

ESP32-Based Real-Time Healthcare Monitoring and Control System

Submitted as part of the Internship Project

Platform: ESP32 + Wokwi

Communication: Wi-Fi / MQTT

Cloud Platform: Adafruit IO

Framework: Arduino + FreeRTOS

Student Name: SANJANA J

Abstract

The Smart Hospital IoT System is an ESP32-based healthcare monitoring and automation platform developed to demonstrate real-time patient monitoring, environmental monitoring, remote control, alert handling, adaptive monitoring, and fault-tolerant communication. The project combines FreeRTOS multitasking with Wi-Fi and MQTT communication to provide concurrent sensor processing, cloud connectivity, local OLED feedback, and safety alerts. Adafruit IO is used as the cloud platform for monitoring and remote control through logical dashboards. The internship requirements are organized into six integrated tasks: multi-sensor expansion using FreeRTOS, dual-role monitoring dashboards, smart medication dosage adjustment, intelligent bed elevation, dynamic sampling, and advanced offline detection. The project is simulated using Wokwi and uses an ESP32 DevKit, temperature sensor, motion sensor, potentiometer, DIP switches, OLED display, LEDs, and buzzers.

1. Introduction

Healthcare environments require continuous monitoring of patient conditions and the surrounding environment. Manual monitoring alone can introduce delays in detecting abnormal conditions. IoT technology can provide continuous data collection, remote monitoring, automated alerts, and control mechanisms. This project demonstrates these concepts through a Smart Hospital IoT System implemented around an ESP32 microcontroller.

The system is designed as a modular real-time platform. Patient-related parameters and environmental conditions are monitored by the controller, processed through FreeRTOS tasks, displayed locally, and published to the cloud using MQTT. Remote control functions are represented through dashboard inputs, while local LEDs, buzzers, and the OLED provide immediate feedback.

2. Problem Statement

A hospital monitoring system must continuously observe patient and environmental conditions, identify abnormal situations quickly, notify responsible personnel, and remain responsive even when multiple operations occur simultaneously. A further challenge is maintaining useful operation during temporary network or MQTT failures. The project addresses these requirements through concurrent processing, cloud-based monitoring, safety logic, adaptive sampling, and fault-tolerant design.

3. Objectives

- Develop an ESP32-based healthcare monitoring and control system.
- Implement concurrent operations using FreeRTOS tasks.
- Use queues and synchronization mechanisms for inter-task communication.
- Monitor patient and environmental parameters.
- Provide separate logical dashboards for medical and facility-related information.
- Implement medication dosage control with safety thresholds and alerts.
- Implement remote bed-angle control with safe operating constraints.
- Provide adaptive monitoring through configurable sampling intervals.
- Detect Wi-Fi/MQTT failures and support offline data handling.
- Provide local feedback through an OLED display, LEDs, and buzzers.
- Demonstrate the complete system using Wokwi simulation and Adafruit IO.

4. System Architecture

The system follows a layered IoT architecture. Sensors and user inputs form the input layer. The ESP32 forms the processing and control layer and executes multiple FreeRTOS tasks. Wi-Fi and MQTT provide the communication layer, while Adafruit IO provides cloud storage, visualization, and dashboard-based control. Local actuators and the OLED provide immediate feedback.

4.1 Major Components

- ESP32 DevKit C / ESP32 microcontroller
- DS18B20 temperature sensor
- PIR motion sensor
- Slide potentiometer for analog control
- DIP switches for simulated patient parameters and control inputs
- 128×64 SSD1306 OLED display
- LED indicators
- Buzzers for abnormal-condition alerts
- Wi-Fi connectivity
- MQTT communication
- Adafruit IO dashboards

4.2 Software Architecture

- Arduino C/C++ firmware
- FreeRTOS tasks for concurrent operations
- FreeRTOS queues for transferring sensor data
- Task delays using vTaskDelay for non-blocking scheduling
- Watchdog-based reliability monitoring
- Adafruit MQTT Library for cloud communication
- DallasTemperature and OneWire libraries for temperature sensing
- Adafruit SSD1306 and GFX libraries for OLED display

5. Hardware Configuration

Component	ESP32 Connection
DS18B20 temperature sensor	GPIO 14
PIR motion sensor	GPIO 12
Control potentiometer	GPIO 34
OLED SDA	GPIO 21
OLED SCL	GPIO 22
Buzzer 1	GPIO 4
Buzzer 2	GPIO 16
Buzzer 3	GPIO 17
Buzzer 4	GPIO 5
DIP switch – motion	GPIO 26
DIP switch – SpO ₂	GPIO 25
DIP switch – heart rate	GPIO 33
DIP switch – temperature	GPIO 32

6. Task 1 – Multi-Sensor Expansion Using FreeRTOS

The first task focuses on concurrent healthcare monitoring. Multiple operations are separated into FreeRTOS tasks so that sensor acquisition and processing do not have to depend on a single blocking loop. Sensor values are transferred through FreeRTOS queues. Priority-based scheduling is used to allow time-sensitive processing to receive processor time appropriately.

The implementation uses a sensor task for acquiring simulated patient parameters and a processing task for evaluating the values, updating the OLED, activating alerts, and communicating with the MQTT broker. The design can be extended with additional environment, communication, alert, and display tasks as the integrated system grows.

7. Task 2 – Dual-Role IoT Monitoring System

The monitoring system separates information logically according to the intended user. Medical staff require patient-vital information, while facility management requires environmental information. MQTT feeds provide separate channels for the corresponding datasets.

- Medical monitoring: heart rate, SpO₂, and body temperature.
- Facility monitoring: room temperature, oxygen level, and AQI.
- Alert/status information is published through dedicated alert feeds.
- Dashboard widgets can display current values and predefined safe ranges.
- Periodic aggregation can be used to summarize monitoring information.

8. Task 3 – Smart Medication Dosage Adjustment

A dashboard slider is used to represent remote medication dosage adjustment in the range of 0–100 mg/hr. The selected dosage is displayed locally on the OLED and evaluated against predefined safety levels.

Dosage Range	Safety Level
0–50 mg/hr	Normal
51–80 mg/hr	Warning
>80 mg/hr	Critical

The critical range activates a red LED and buzzer to provide an immediate local warning. Rate limiting is used to reduce abrupt dosage changes, and critical values can be handled through an additional confirmation or safety check.

9. Task 4 – Intelligent Remote Bed Elevation

The bed-control module represents remote adjustment of the hospital bed angle from 0° to 90°. The intended implementation uses a dashboard control and a servo mechanism. Movement is designed to be gradual rather than abrupt, reducing sudden changes in the simulated bed position.

Preset Mode	Angle
Sleeping	10°
Breathing Support	45°
Emergency	90°

Automatic adjustment logic can use patient-condition information to select an appropriate bed position. Safety constraints limit sudden or invalid angle changes.

10. Task 5 – Smart Dynamic Sampling Rate

The monitoring frequency is configurable from 5 to 60 seconds. A manual sampling value can be selected from the dashboard, while automatic operation can increase the sampling frequency when abnormal patient conditions are detected.

- High-frequency monitoring: 5-second interval during abnormal conditions.
- Stable monitoring: longer interval when conditions remain normal.
- FreeRTOS vTaskDelay is used for non-blocking task scheduling.
- Sampling configuration can be communicated through queues or shared state.

Adaptive sampling reduces unnecessary processing and communication during stable conditions while allowing faster observation during abnormal conditions.

11. Task 6 – Advanced Offline Detection

The fault-tolerant design monitors Wi-Fi and MQTT connectivity and classifies the system into ONLINE, DEGRADED, or OFFLINE states. When connectivity is unavailable, the system provides local feedback instead of silently stopping operation.

- ONLINE: normal Wi-Fi and MQTT operation.
- DEGRADED: partial communication or service problems.
- OFFLINE: communication unavailable and local operation continues.
- OLED displays 'LOGGING OFFLINE' during offline operation.
- Distinct buzzer patterns can identify communication failures.
- Sensor readings can be placed into a local queue/buffer.
- Buffered readings can be synchronized after connectivity is restored.
- Retry logic with increasing delays can reduce repeated connection attempts.

12. FreeRTOS and Inter-Task Communication

FreeRTOS is central to the real-time behavior of the project. Tasks allow sensor acquisition, processing, communication, alert generation, and display updates to execute independently. Queues provide a controlled method of transferring data between tasks. Semaphores and mutexes can be used where shared resources require synchronization. The watchdog timer provides an additional reliability mechanism by detecting tasks that stop responding.

13. MQTT and Cloud Communication

The ESP32 connects to Wi-Fi and communicates with Adafruit IO through MQTT. MQTT follows a publish/subscribe model, allowing sensor data and alert states to be sent to separate feeds. Dashboard controls can similarly be represented as remote input values received by the controller.

- Body temperature feed
- Heart-rate feed
- SpO₂ feed
- Blood-pressure feed where configured
- Room-temperature feed

- Oxygen-level feed
- AQI feed
- Alert/status feed
- Medication dosage feed where configured

14. OLED Display and Alert Handling

The SSD1306 OLED provides local visibility of important system values and alerts. The display can show patient parameters, medication status, connectivity state, and abnormal-condition messages.

- Temperature status
- Heart-rate status
- SpO₂ status
- Medication status
- Connectivity/offline status
- System alert messages

Multiple buzzers and LED indicators are used to provide immediate local notification for abnormal conditions. Different outputs can be assigned to different alert categories.

15. Testing and Expected Results

The system is tested using Wokwi simulation inputs and dashboard controls. Sensor values are varied to verify threshold detection, display updates, alert generation, and MQTT publication. Control values are varied to verify dosage and bed-control safety logic. Connectivity failure scenarios are used to verify offline-state handling.

Test Case	Input / Condition	Expected Result
High temperature	Temperature above threshold	Temperature alert and local notification
Low heart rate	Heart rate below threshold	Heart-rate warning
Low SpO ₂	SpO ₂ below threshold	SpO ₂ warning
Motion detection	PIR detects motion	Motion alert
Critical dosage	Dosage >80 mg/hr	Red LED + buzzer + critical status
Sampling adaptation	Abnormal vital detected	Higher monitoring frequency
Wi-Fi/MQTT failure	Connectivity lost	Offline/degraded indication
Recovery	Connectivity restored	Normal communication resumes

16. Challenges Faced and Solutions

Challenge	Issue	Approach
Concurrent task execution	Running sensor monitoring, MQTT communication, alert handling, and display updates without blocking.	Separated operations into FreeRTOS tasks and used task delays and queues for communication.
MQTT connectivity	Maintaining cloud communication while the system continues processing local inputs.	Connection checks and MQTT processing are performed as part of the communication logic.
Safety of remote controls	Sudden medication or bed-position changes can produce unsafe simulated behavior.	Applied thresholds, rate limiting, constrained ranges, and preset operating modes.

System reliability	A blocked task can affect the responsiveness of the whole controller.	Used FreeRTOS scheduling and watchdog-based monitoring.
Network failure	Sensor data may be lost when Wi-Fi or MQTT is unavailable.	Offline state detection and local buffering/synchronization are used in the fault-tolerant design.

17. Advantages

- Real-time monitoring and alert generation
- Concurrent execution using FreeRTOS
- Remote cloud-based monitoring
- Logical separation of medical and facility information
- Safety-oriented remote control
- Adaptive monitoring frequency
- Local feedback through OLED, LEDs, and buzzers
- Fault-tolerant operation during connectivity problems
- Scalable MQTT-based architecture

18. Limitations

- The project is primarily a simulation/prototype and is not intended for clinical use.
- Several patient parameters are simulated using Wokwi inputs rather than clinical-grade sensors.
- Actual hospital deployment would require validated medical sensors and regulatory compliance.
- Cloud connectivity depends on Wi-Fi and MQTT availability.
- The simulation does not reproduce all physical characteristics of real hospital equipment.

19. Future Scope

- Integration of clinical-grade heart-rate, SpO₂, ECG, and blood-pressure sensors.
- Integration with hospital information systems and electronic health records.
- Secure MQTT communication using TLS and stronger authentication.
- Database-based long-term patient history and analytics.
- Machine-learning-based anomaly detection.
- Mobile notifications for critical events.
- Physical smart-bed and infusion-pump integration.
- Role-based authentication and access control.
- More advanced offline storage using SPIFFS or non-volatile memory.

20. Conclusion

The Smart Hospital IoT System demonstrates how an ESP32 and FreeRTOS can be used to build a real-time healthcare monitoring and automation prototype. The project combines patient monitoring, environmental monitoring, cloud dashboards, medication control, bed control, adaptive sampling, and fault-tolerant communication into a single IoT architecture. The use of MQTT provides a scalable communication mechanism, while FreeRTOS enables concurrent execution of time-sensitive operations. The Wokwi

implementation provides a safe environment for testing the system logic before considering physical deployment.

21. Appendix – Software Libraries

- Adafruit GFX Library
- OneWire
- DallasTemperature
- Adafruit SSD1306
- Adafruit MQTT Library
- ESP32 FreeRTOS APIs

22. Appendix – Suggested Figures

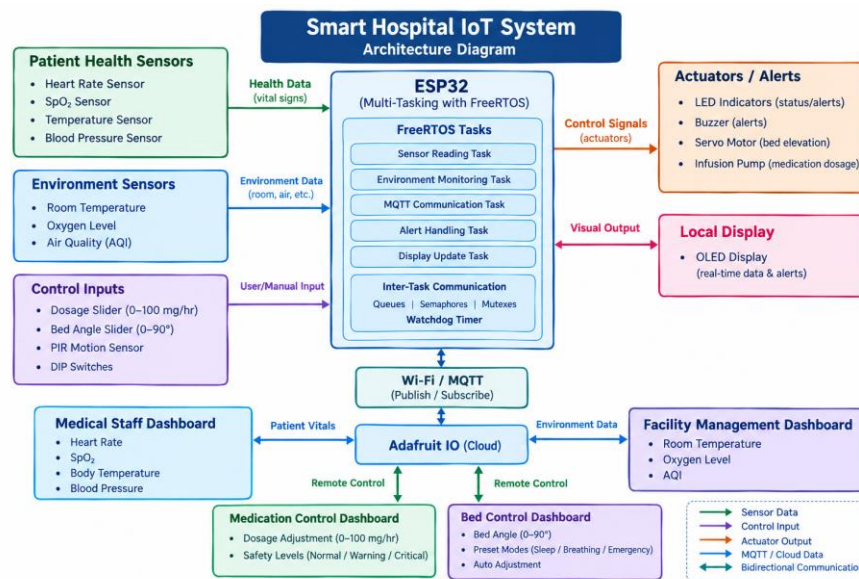


Fig 1: Architecture Diagram

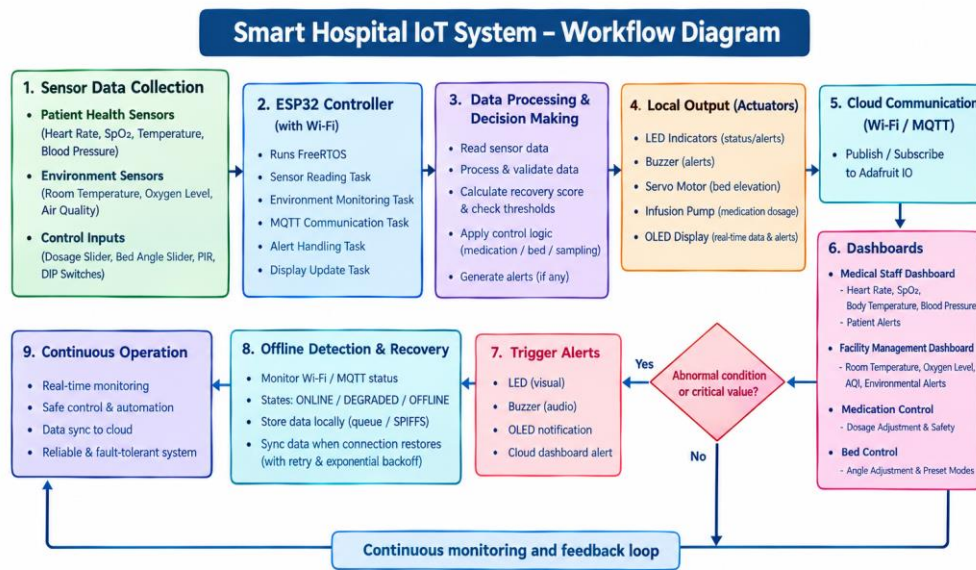


Fig 2: Workflow Diagram

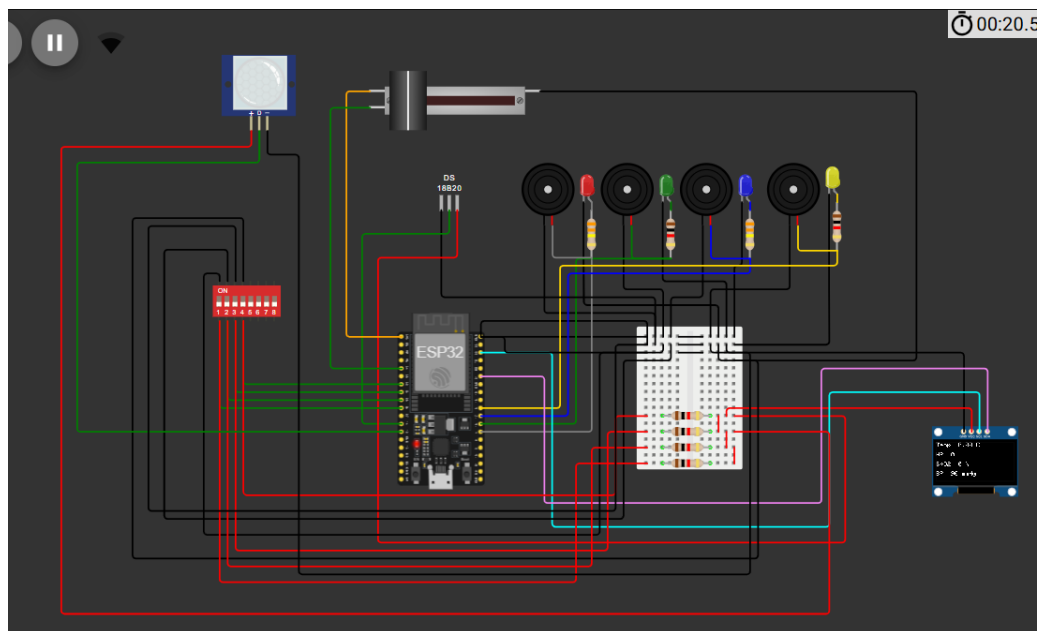


Fig 3: Circuit Diagram

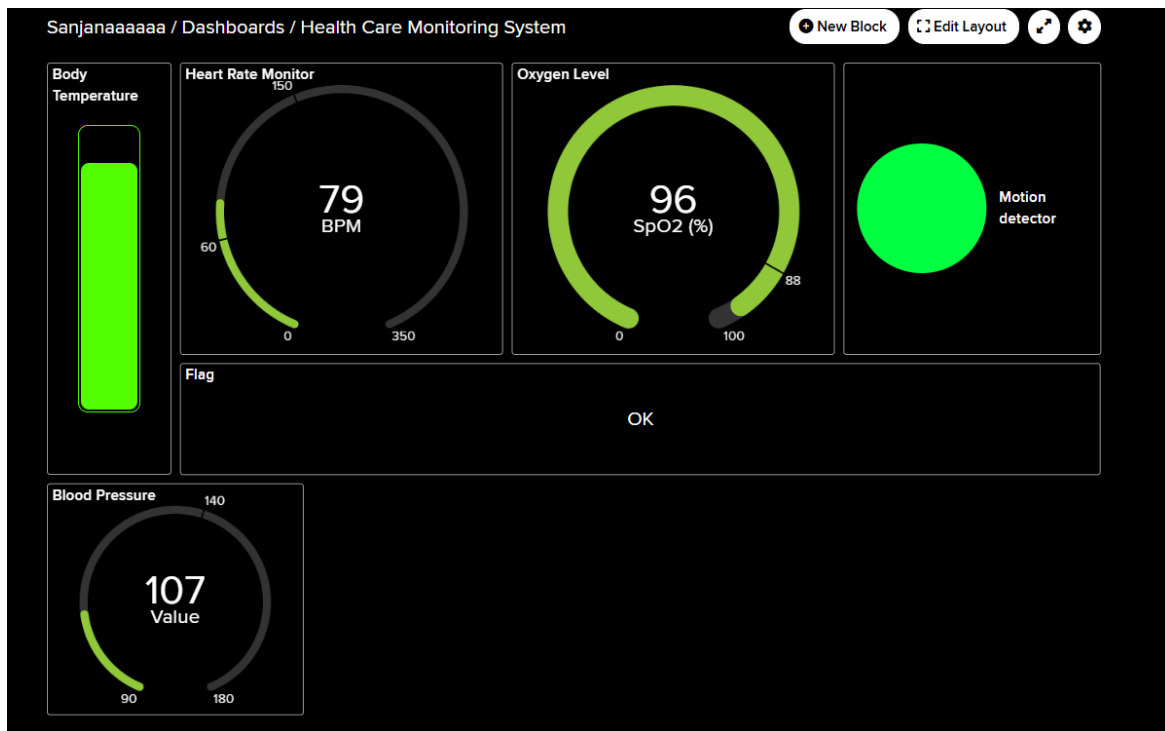


Fig 4: Patient Dashboard

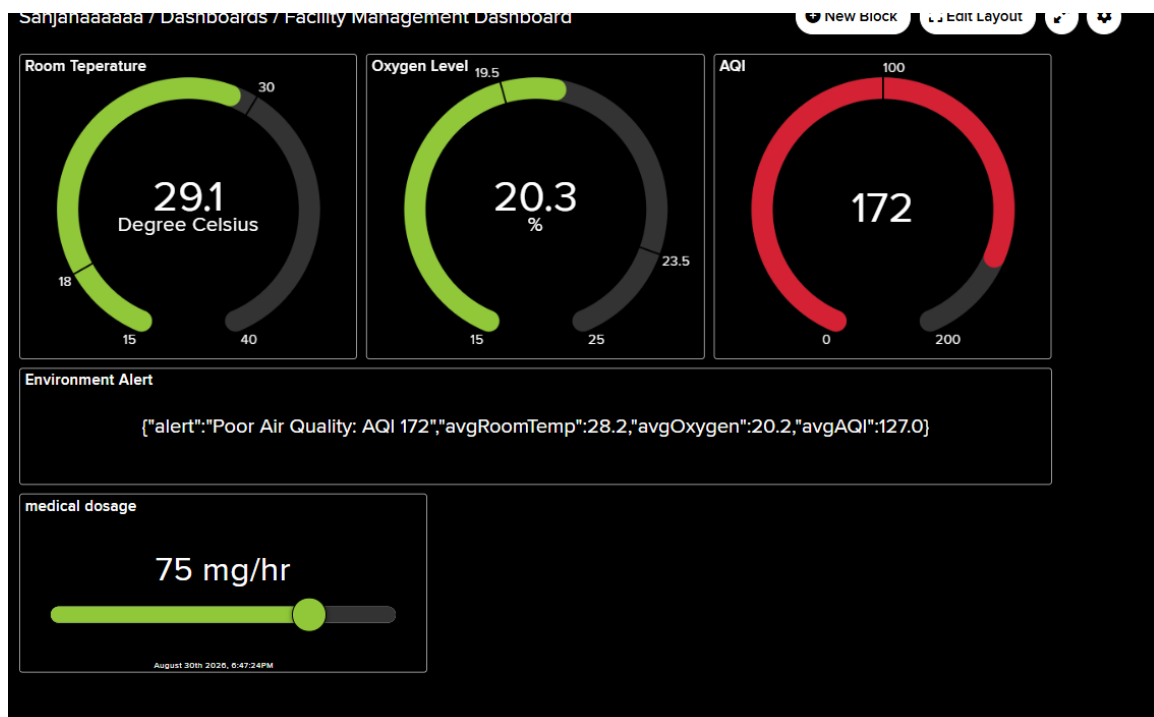


Fig 5: Facility Dashboard